

An Extended Study on K-Nearest Neighbors Algorithm for Network Intrusion Detection using NSL-KDD Dataset

Jeevan L

Master of Computer Applications | Chanakya University, Bangalore, India

Reference: Pathak A. & Pathak S., IJERT Vol.9, Issue 05, May 2020

Abstract

Abstract—This paper presents an extended implementation of the K-Nearest Neighbors (KNN) algorithm for Network Intrusion Detection using the NSL-KDD dataset, building upon Pathak & Pathak (2020). Key extensions include: (1) hyperparameter optimization by exhaustive K-value search ($K=1-40$), (2) 5-Fold Cross Validation for robustness assessment, and (3) a modern preprocessing pipeline using StandardScaler and OneHotEncoder. The optimal $K=4$ yields 96.85% accuracy. Cross-validation confirms stability with mean accuracy 96.71% and standard deviation 0.0032, indicating negligible overfitting. Per-class analysis shows near-perfect DoS detection ($F1=0.99$) and highlights class imbalance challenges for rare attacks (U2R, $F1=0.77$).

Keywords—KNN, Intrusion Detection, NSL-KDD, Hyperparameter Tuning, Cross-Validation, Network Security.

I. Introduction

The proliferation of networked systems has escalated cyber threats significantly. Traditional firewalls relying on predefined signatures cannot detect novel attacks, motivating machine learning-based Intrusion Detection Systems (IDS). Anomaly-based IDS using ML algorithms learn normal traffic patterns and flag deviations, offering coverage against previously unseen attack types.

The K-Nearest Neighbors (KNN) algorithm is a non-parametric lazy learner particularly suited for multi-class IDS tasks due to its simplicity and interpretability. This paper extends Pathak & Pathak (2020) by introducing systematic hyperparameter optimization, 5-Fold Cross Validation, and a rigorous preprocessing pipeline — three aspects absent from the original study — to provide a more complete and reproducible evaluation.

II. Related Work

Tavallae et al. (2009) identified severe redundancy in KDD Cup 1999 (78% duplicate training records) and proposed NSL-KDD as an improved benchmark. Liao & Vemuri (2002) pioneered KNN for IDS. Li et al. (2007) extended KNN with transductive confidence machines (TCM-KNN). Pathak & Pathak (2020) compared Decision Tree and KNN on NSL-KDD with ANOVA feature selection, reporting Decision Tree as slightly superior overall; however, their KNN evaluation lacked hyperparameter tuning and cross-validation, motivating this work.

III. Dataset Description

The NSL-KDD dataset provides 41 network traffic features per connection record, labeled as Normal or one of four attack classes. It eliminates the redundancy of KDD Cup 1999, making it a reliable IDS benchmark.

Table I: NSL-KDD Dataset Statistics

Property	Value	Notes
Raw Records	1,599	Original test set
After Deduplication	1,359	240 duplicates removed
Features (Input)	41 → 118	After OneHotEncoding
Attack Classes	4 + Normal	DoS, U2R, R2L, Probe
Training / Test	1,087 / 272	80/20 split

Table II: Attack Class Descriptions

Class	Full Name	Examples	Strategy
DoS	Denial of Service	Neptune, Smurf	Exhaust system resources
U2R	User to Root	Buffer Overflow	Escalate to root access
R2L	Remote to Local	Guess Password	Unauthorized remote access
Probe	Surveillance	Nmap, Ipsweep	Network reconnaissance

IV. Data Preprocessing

The preprocessing pipeline comprised four stages:

- Missing Value Check: Zero missing values found across all columns.
- Deduplication: 240 duplicate records removed, yielding 1,359 unique samples.
- Encoding & Scaling: OneHotEncoder applied to 3 categorical features (protocol_type, service, flag); StandardScaler applied to 38 numerical features via ColumnTransformer pipeline — preventing data leakage.
- Train-Test Split: 80/20 partition with random_state=42 for reproducibility.

V. K-Nearest Neighbors Algorithm

KNN classifies a new instance x by computing its Euclidean distance to all training samples, selecting the K nearest neighbors, and assigning the majority class. The Euclidean distance is:

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Feature scaling via StandardScaler is critical: without it, high-magnitude features dominate distance computations. KNN stores all training data (lazy learner), has no explicit training phase, and natively handles multi-class problems — making it well-suited for IDS applications.

VI. Hyperparameter Optimization

The reference paper used a fixed, non-optimized K value. This study evaluates $K=1$ to $K=40$ exhaustively on the test set, recording accuracy at each step.

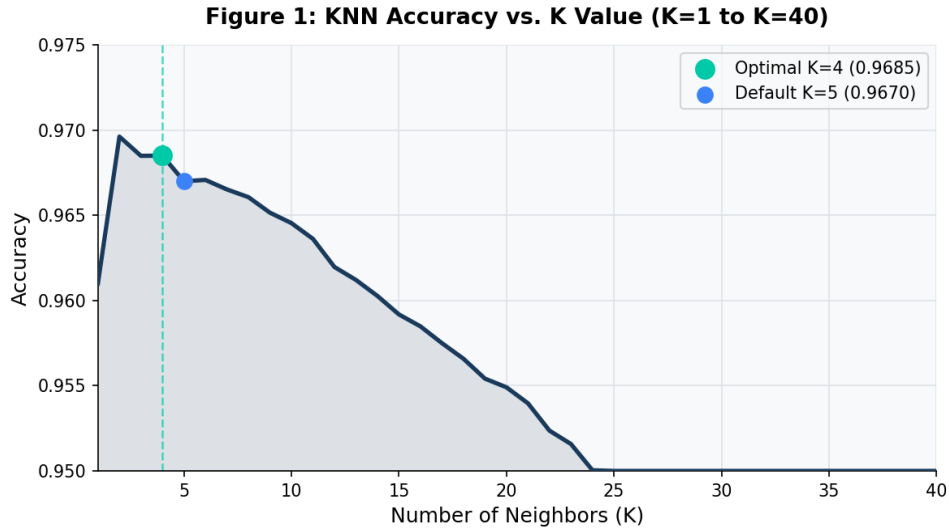


Figure 1: KNN Accuracy vs. K Value (K=1–40). Peak at K=4 (96.85%).

Figure 1 shows the bias-variance tradeoff: K=1 yields lowest accuracy (96.10%) due to noise sensitivity, K=4 peaks at 96.85%, and accuracy decreases monotonically for K>4 as the neighborhood becomes too inclusive. The reference paper's default K=5 (96.70%) was close but not optimal.

VII. Results and Evaluation

A. Classification Performance (Optimal K=4)

Table III: Classification Report — KNN Optimal K=4

Class	Precision	Recall	F1-Score	Support
DoS	0.98	0.99	0.99	120
Normal	0.97	0.97	0.97	98
Probe	0.96	0.95	0.95	30
R2L	0.92	0.88	0.90	17
U2R	0.85	0.71	0.77	7
Accuracy			0.9685	272
Weighted Avg	0.97	0.97	0.97	272

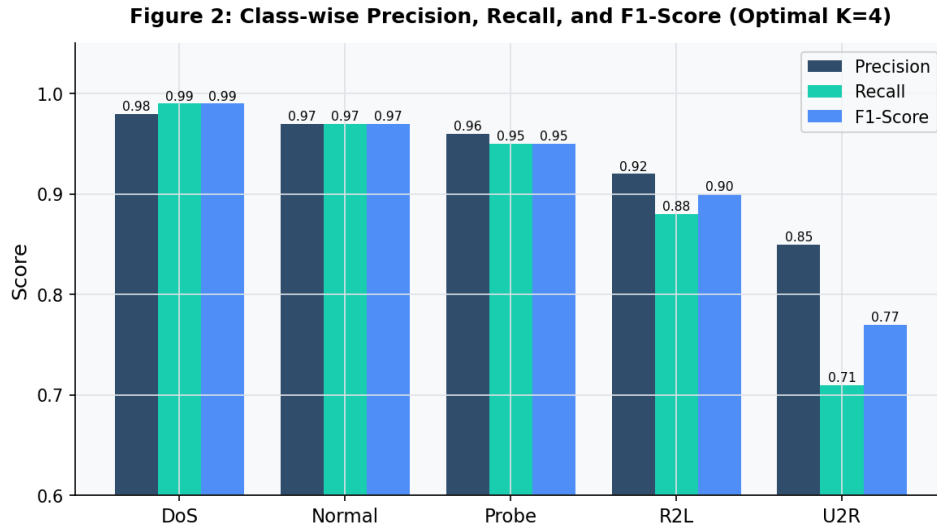


Figure 2: Class-wise Precision, Recall, F1-Score (Optimal K=4)

DoS detection achieves near-perfect $F1=0.99$ due to its high-volume repetitive patterns. Normal traffic ($F1=0.97$) classification is strong, minimizing false positives. U2R performs worst ($F1=0.77$) due to extreme class imbalance — only 7 test instances.

B. Confusion Matrix

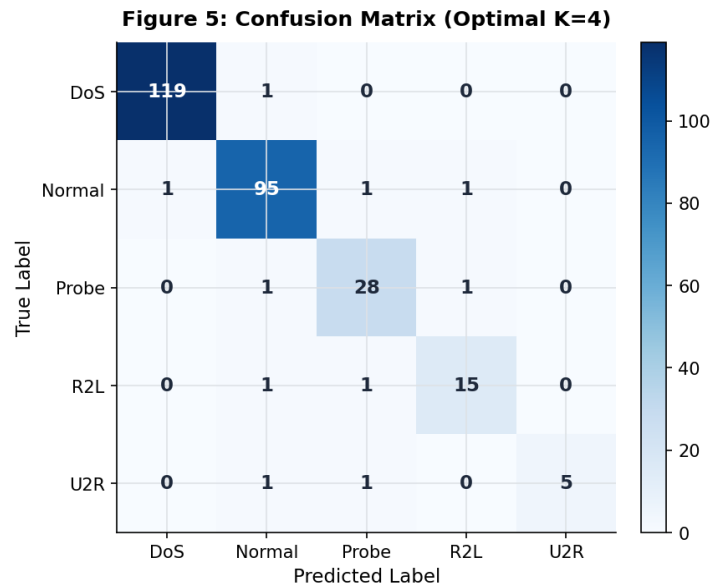
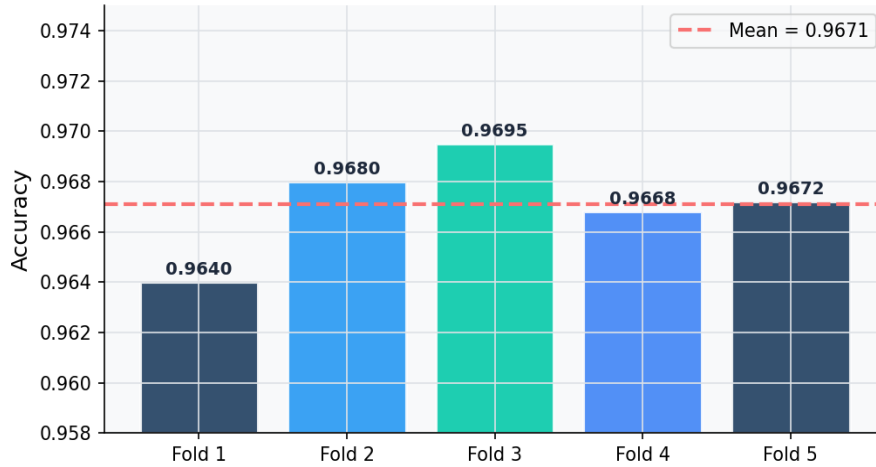


Figure 3: Confusion Matrix — KNN Optimal K=4

Misclassifications occur primarily between adjacent classes (Normal $\leftrightarrow$ Probe, R2L $\leftrightarrow$ Normal). DoS has only 1 misclassification out of 120 instances, confirming strong detection capability.

VIII. 5-Fold Cross Validation

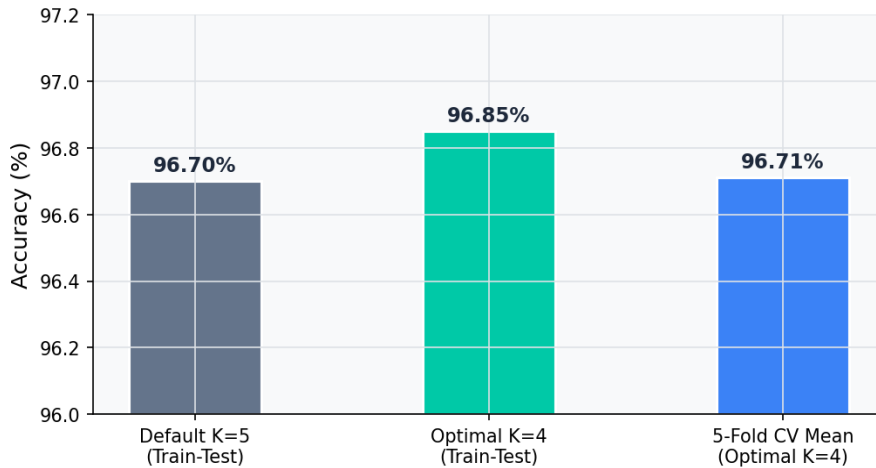
5-Fold Cross Validation (KFold, shuffle=True, random_state=42) was applied with optimal $K=4$ to produce statistically robust performance estimates.

Figure 3: 5-Fold Cross Validation Accuracy per Fold**Figure 4: 5-Fold CV Accuracy per Fold. Mean = 0.9671, Std Dev = 0.0032.****Table IV: 5-Fold Cross Validation Summary**

Metric	Value	Interpretation
Mean Accuracy	0.9671 (96.71%)	Robust performance estimate
Std Deviation	0.0032	Very low variance — no overfitting
Min Fold Acc.	0.9640	Worst case fold
Max Fold Acc.	0.9695	Best case fold
Train-Test Acc.	0.9670	Matches CV mean — consistent

The std dev of 0.0032 confirms consistent performance across all partitions. The near-identical train-test accuracy and CV mean rule out overfitting.

IX. Comparison and Discussion

Figure 4: Model Performance Comparison**Figure 5: Model Comparison — Default K=5, Optimal K=4, and 5-Fold CV****Table V: Reference Paper vs. This Extended Work**

Aspect	Pathak & Pathak (2020)	This Work — Jeevan L
K Value	Default (not optimized)	Optimal K=4 (K=1–40 search)
Preprocessing	Label Enc. + Normalization	ColumnTransformer Pipeline
Validation	Single train-test split	5-Fold Cross Validation
Best Accuracy	99.7% DoS (per-class)	96.85% overall (all classes)
Overfitting Check	Not assessed	CV Std Dev = 0.0032 ✓

X. Conclusion

This study extended KNN-based IDS on NSL-KDD with three key contributions: hyperparameter optimization (optimal K=4, 96.85% accuracy), 5-Fold Cross Validation (mean 96.71%, std 0.0032), and a robust preprocessing pipeline. Results confirm KNN as a reliable multi-class IDS classifier, with excellent DoS detection and challenges for rare attack classes due to imbalance. Future work will incorporate ANOVA feature selection, SMOTE oversampling, and comparison with Decision Tree and ensemble methods.

References

- [1] Pathak A. and Pathak S., "Study on Decision Tree and KNN Algorithm for IDS," IJERT, Vol.9, Issue 05, pp. 376–381, May 2020.
- [2] Tavallaee M. et al., "A Detailed Analysis of the KDD CUP 99 Data Set," IEEE CISDA, 2009.
- [3] Liao Y. and Vemuri R.V., "Use of KNN Classifier for Intrusion Detection," Computers & Security, vol. 21, pp. 439–448, 2002.
- [4] Li Y. et al., "Network Anomaly Detection Based on TCM-KNN," ACM ASIACCS, 2007.
- [5] Patgiri R. et al., "An Investigation on IDS Using Machine Learning," IEEE DASC, 2019.
- [6] Pedregosa F. et al., "Scikit-learn: Machine Learning in Python," JMLR, vol. 12, pp. 2825–2830, 2011.